



**KLE Technological
University** | Creating Value,
Leveraging Knowledge

**Department
of
Electronics and Communication Engineering**

**Minor Project Report
on**

**PERFORMANCE ANALYSIS OF COHERENCE PROTOCOLS
FOR AN 8-CORE TILED ARCHITECTURE BASED ON KEY
EVALUATION PARAMETERS**

By:

- | | |
|----------------------|-------------------|
| 1. Pavitra Kambar | USN: 01FE23BEC379 |
| 2. Sristi Bhat | USN: 01FE23BEC146 |
| 3. Saraswati Bhakta | USN: 01FE23BEC102 |
| 4. Aishwarya Kumbare | USN: 01FE23BEC297 |

Semester: VI, 2025-2026

**Under the Guidance of
Prof. Jayashree M & Prof. Rajeshwari M**

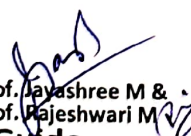
K.L.E SOCIETY'S
KLE Technological University,
HUBBALLI-580031
2025-26



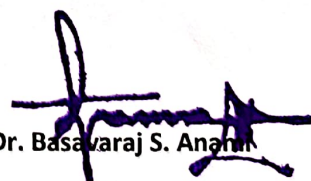
DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING

CERTIFICATE

This is to certify that project entitled "Performance Analysis of Coherence Protocols for an 8-Core Tiled Architecture Based on Key Evaluation Parameters" is a bonafide work carried out by the student team of "Saraswati (01FE23BEC102), Sristi (01FE23BEC146), Pavitra (01FE23BEC379), and Aishwarya (01FE23BEC297)". The project report has been approved as it satisfies the requirements with respect to the work prescribed by the university curriculum for BE (VI Semester) in the Department of Electronics and Communication Engineering of KLE Technological University for the academic year 2025-2026.


Prof. Jayashree M &
Prof. Rajeshwari M
Guide


Dr. Suneeta Budihal
Head of Department


Dr. Basavaraj S. Ananth
Registrar

External Viva:

Name of Examiners

1. Saroja V. S
2. H. Anant

 6/4/24-
Signature with date

 6/4/26

ACKNOWLEDGMENT

We express our sincere gratitude to our project guide, Prof. Jayashree M, for her invaluable guidance, continuous encouragement, suggestions, and support throughout the course of this project. Her expertise, patience, and motivation have been instrumental in the successful completion of our work.

We extend our heartfelt thanks to Dr. Suneeta Bhudihal, Head of the Department of Electronics and Communication Engineering, for her constant encouragement, valuable support, and for providing the facilities and academic environment necessary to carry out this project successfully.

We also express our sincere appreciation to the faculty members and staff of the Department of Electronics and Communication Engineering, KLE Technological University, for their guidance, encouragement, and technical assistance throughout the project.

We are grateful to KLE Technological University for providing excellent laboratory facilities, computing resources, and an enriching academic environment that enabled us to successfully complete this project.

We sincerely thank everyone who contributed to the successful completion of our project entitled "Performance Analysis of Coherence Protocols for an 8-Core Tiled Architecture Based on Key Evaluation Parameters".

ABSTRACT

Modern tiled chip multiprocessor (TCMP) architectures rely on efficient cache coherence mechanisms to ensure data consistency among multiple processor cores while minimizing memory access latency. The MESI (Modified, Exclusive, Shared, and Invalid) cache coherence protocol is one of the most widely adopted protocols for maintaining coherence in multicore systems due to its simplicity and effective state management. However, the conventional MESI protocol does not exploit neighbour locality during cache-to-cache communication. As a result, memory requests may follow longer coherence paths even when the required data is available in a neighbouring cache, leading to increased memory access latency and additional coherence traffic.

To address this limitation, this work proposes a Neighbour-Aware IN (In-Neighbour) cache state as an extension to the conventional MESI protocol. The proposed IN state enables a processor core to identify valid data available in neighbouring caches and retrieve it directly whenever possible, thereby reducing unnecessary memory accesses and improving communication efficiency. By utilizing neighbour locality within the tiled architecture, the proposed approach minimizes communication overhead while preserving cache coherence correctness.

The modified MESI protocol with the IN state has been implemented and evaluated using the gem5 simulator with the Ruby memory system on an 8-core tiled multicore architecture employing the SimplePt2Pt interconnection topology. Performance analysis has been carried out using five PARSEC benchmark applications. The proposed protocol is evaluated using key performance metrics such as simulation time, simulated operations, simulated instructions, IPC, CPI, cache hit rate, cache miss rate, average latency, network traffic, and Average Memory Access Time (AMAT). The experimental results demonstrate that the proposed neighbour-aware mechanism improves processor performance by reducing memory access latency and communication overhead while maintaining cache coherence. Furthermore, the study discusses the scalability limitations of the SimplePt2Pt topology and highlights Mesh_XY as a promising interconnection topology for future implementation in larger multicore systems.

Contents

Chapter 1 Introduction	9
1.1 Objectives.....	10
1.2 Literature Survey.....	10
1.3 Problem Statement	11
1.4 Application in Societal Context	11
Chapter 2 Neighbour-Aware Extension of the MESI Cache Coherence Protocol for Reducing Memory Access Latency in an 8-Core Tiled Multicore Architecture	12
2.1 Tiled Chip Multicore Processor Architecture	12
2.2 Cache Coherence in TCMPs and Baseline MESI Protocol.....	13
2.3 Identification of Neighbour Locality Limitation	14
2.4 Design of the Neighbour-Aware IN State	15
2.5 Extension of the MESI State Machine	15
2.6 Design Specifications and Implementation in the gem5 Ruby Memory System	16
2.7 Experimental Setup and Evaluation Methodology	17
2.8 Workloads Used	18
2.9 Classification of PARSEC Benchmark Applications.....	19
2.10 Performance Metrics Calculation.....	20
Chapter 3 Results and Discussions.....	21
3.1 Performance Metrics.....	21
3.2 Processor Efficiency Metrics	22
3.3 Memory and Network Metrics.....	22
3.4 Overall Discussion	23
Chapter 4 Conclusion.....	24
Future Scope	24
References.....	25

List of Tables

Table 2.1 System Configuration considered for experimentation.....	17
--	----

List of Figures

Figure 2.1 Internal view of Tiled Multicore Processor Architecture.....	13
Figure 2.2 MESI Cache Coherence State Transition Diagram	14
Figure 2.3: MESI+IN Cache Coherence State Transition Diagram.....	16
Figure 2.4 : 8-Core tiled multicore architecture implemented in the gem5 simulator using the SimplePt2Pt interconnection topology.....	18
Figure 3.1 Performance Comparison of the Existing MESI Protocol and the Proposed MESI+IN Protocol .	21
Figure 3.2 Comparison of Processor Efficiency Metrics for the Existing and Proposed Protocols	22
Figure 3.3 Comparison of Memory and Network Performance Metrics for the Existing and Proposed Protocols	22

Chapter 1

Introduction

The rapid advancement of semiconductor technology has enabled the integration of multiple processing cores onto a single chip, leading to the development of multicore processor architectures. These architectures have become the foundation of modern computing systems due to their ability to execute multiple tasks simultaneously, thereby improving computational performance and energy efficiency. As the number of processing cores continues to increase, efficient communication and data sharing among the cores have become important challenges in achieving high system performance.

In multicore processors, each core is generally equipped with private cache memories to reduce memory access latency and improve execution speed. Since multiple cores may access or modify the same memory location simultaneously, maintaining data consistency across all caches becomes essential. Cache coherence protocols ensure that every processor core observes the most recent and consistent copy of shared data while minimizing communication overhead. Among the commonly used cache coherence protocols, the Modified, Exclusive, Shared, and Invalid (MESI) protocol is one of the most widely adopted because of its simplicity and efficient coherence management. The MESI protocol maintains data consistency by allowing cache blocks to transition between four stable states based on processor read and write operations. Although the protocol performs well in multicore systems, it does not explicitly consider neighbour locality during cache-to-cache communication. Consequently, even when the requested data is available in a nearby cache, the conventional coherence mechanism may still follow longer communication paths, resulting in increased memory access latency and additional coherence traffic.

To address this limitation, this work introduces an additional cache state called IN (In-Neighbour) into the conventional MESI protocol. The proposed neighbour-aware state enables processor cores to identify valid data available in neighbouring caches and retrieve it directly whenever possible. By utilizing neighbour locality, the proposed approach reduces unnecessary memory accesses, decreases communication latency, and improves overall cache utilization while preserving coherence correctness. The proposed neighbour-aware MESI protocol has been implemented and evaluated using the gem5 simulator together with the Ruby memory system on an 8-core tiled multicore architecture employing the SimplePt2Pt interconnection topology. Performance evaluation has been carried out using five PARSEC benchmark applications. Various performance parameters, including Average Memory Access Time (AMAT), Instructions Per Cycle (IPC), Cycles Per Instruction (CPI), execution time, queueing latency and network latency have been analysed to compare the proposed protocol with the conventional MESI protocol. The study further discusses the limitations of the SimplePt2Pt topology and highlights the advantages of adopting more scalable interconnection networks, such as Mesh_{XY}, for future multicore architectures.

1.1 Objectives

The primary objectives of this work are:

- To study the behaviour of the conventional MESI cache coherence protocol in an 8-core tiled multicore architecture.
- To design MESI coherence protocol with new state called IN(In Neighbour) state for 8 core tiled architecture.
- To evaluate the performance of baseline and proposed protocols using PARSEC benchmark applications.
- To analyse system performance using key evaluation parameters such as IPC, CPI, AMAT, execution time, queueing latency and network latency.

1.2 Literature Survey

The increasing demand for high-performance computing has led to significant research in multicore processor architectures, cache coherence mechanisms, and Network-on-Chip (NoC) communication systems. Tiled multicore processors have emerged as an efficient solution for supporting parallel execution by organizing multiple processing cores into a structured architecture. These processors provide improved scalability, efficient communication, and better resource utilization compared to conventional shared-bus architectures.

Network-on-Chip (NoC) architectures have been widely adopted to facilitate communication among multiple processing cores. Unlike traditional bus-based communication systems, NoC provides scalable communication through interconnected routers and communication links. Several researchers have demonstrated that structured interconnection networks improve communication efficiency and reduce congestion in multicore processors. Among various NoC topologies, SimplePt2Pt provides direct communication between connected components, whereas topologies such as Mesh_XY offer better scalability and organized routing for larger multicore systems.

Cache coherence protocols play an important role in maintaining data consistency among multiple private caches in shared-memory multicore processors. Protocols such as MSI, MESI, and MOESI have been extensively studied to improve communication efficiency while preserving coherence correctness. The MESI protocol remains one of the most commonly adopted coherence protocols because of its relatively simple state machine and effective management of cache states. However, the conventional MESI protocol does not exploit neighbour locality during cache-to-cache communication, which may result in increased memory access latency and unnecessary coherence traffic.

The gem5 simulator has become one of the most widely used simulation platforms for computer architecture research due to its flexibility in modelling multicore processors, cache hierarchies, and memory systems. Its Ruby memory system provides detailed support for implementing and evaluating different cache coherence protocols. Similarly, the PARSEC benchmark suite is extensively used for evaluating multicore processor performance because it contains realistic parallel workloads with diverse computational and memory access characteristics.

Several recent studies have focused on reducing communication latency by exploiting locality-aware communication mechanisms. These approaches demonstrate that utilizing nearby cache resources can significantly reduce memory access latency and improve processor performance without compromising

cache coherence. Inspired by these observations, the present work extends the conventional MESI protocol by introducing a neighbour-aware cache state to improve cache-to-cache communication efficiency.

1.3 Problem Statement

Neighbour-Aware Extension of the MESI Cache Coherence Protocol for Reducing Memory Access Latency in an 8-Core Tiled Multicore Architecture

Modern multicore processors rely on efficient cache coherence protocols to maintain data consistency among multiple processing cores. The MESI protocol is widely used because of its simple implementation and effective management of shared cache data. However, the conventional MESI protocol does not consider neighbour locality during cache-to-cache communication. Regardless of the physical location of the requested data, memory requests follow the standard coherence path, even when the required cache block is available in a nearby processor core.

As the number of processor cores increases, this behaviour results in higher communication overhead, increased memory access latency, and additional coherence traffic. These limitations reduce overall processor efficiency, particularly for applications exhibiting high spatial locality and frequent data sharing between neighbouring cores.

To overcome this limitation, there is a need for a neighbour-aware cache coherence mechanism capable of identifying and retrieving data from neighbouring caches whenever possible while maintaining coherence correctness. Therefore, this work extends the conventional MESI protocol by introducing an additional IN (In-Neighbour) cache state. The proposed neighbour-aware extension aims to reduce unnecessary memory accesses, improve cache utilization, minimize communication latency, and enhance the overall performance of an 8-core tiled multicore architecture implemented using the SimplePt2Pt interconnection topology.

1.4 Application in Societal Context

The proposed neighbour-aware extension of the MESI cache coherence protocol contributes to the development of faster and more efficient multicore computing systems. By reducing memory access latency and improving cache utilization, the proposed approach enhances processor performance while maintaining data consistency among multiple processing cores.

Such improvements are beneficial in several real-world applications, including artificial intelligence, machine learning, cloud computing, scientific simulations, medical image processing, real-time embedded systems, and multimedia applications. Faster communication between processor cores enables quicker execution of computationally intensive tasks, improving system responsiveness and overall productivity.

Furthermore, reducing unnecessary memory accesses and communication overhead contributes to improved energy efficiency, making future multicore processors more suitable for sustainable computing environments. The concepts developed in this work can also be extended to larger multicore architectures and more scalable Network-on-Chip topologies, supporting the continued advancement of high-performance computing systems.

Chapter 2

Neighbour-Aware Extension of the MESI Cache Coherence Protocol for Reducing Memory Access Latency in an 8-Core Tiled Multicore Architecture

This chapter describes the design, implementation, and evaluation of the proposed neighbour-aware extension to the MESI cache coherence protocol for an 8-core tiled multicore processor. The methodology begins with the description of the tiled multicore architecture used in this work, followed by an overview of the baseline MESI protocol and the identification of its limitations in exploiting neighbour locality. The proposed IN (In-Neighbour) state is then introduced to improve cache-to-cache communication by allowing neighbouring caches to participate in data sharing. Finally, the implementation of the modified protocol in the gem5 simulator and the experimental setup used for performance evaluation are presented.

2.1 Tiled Chip Multicore Processor Architecture

The proposed cache coherence enhancement is evaluated on an 8-core tiled multicore processor architecture. In this architecture, eight identical processing tiles are interconnected through a communication network. Each processing tile consists of a processor core, private Level-1 instruction and data caches, and interfaces with the shared memory hierarchy. The tiled organization provides modularity, improved scalability, and efficient communication among processor cores.

Each processor core executes application threads independently while accessing its private cache for frequently used instructions and data. Shared data communication between cores is managed through the cache coherence protocol, ensuring that all processors maintain a consistent view of memory. The system uses the SimplePt2Pt interconnection topology, where communication between components occurs through dedicated point-to-point links. This topology provides straightforward communication for small multicore systems and serves as the baseline interconnection network for evaluating the proposed protocol.

The architecture is modelled using the gem5 simulator together with the Ruby memory system, which enables cycle-accurate simulation of cache coherence behaviour and memory communication. This simulation environment provides a realistic platform for analysing the impact of the proposed neighbour-aware extension on processor performance.

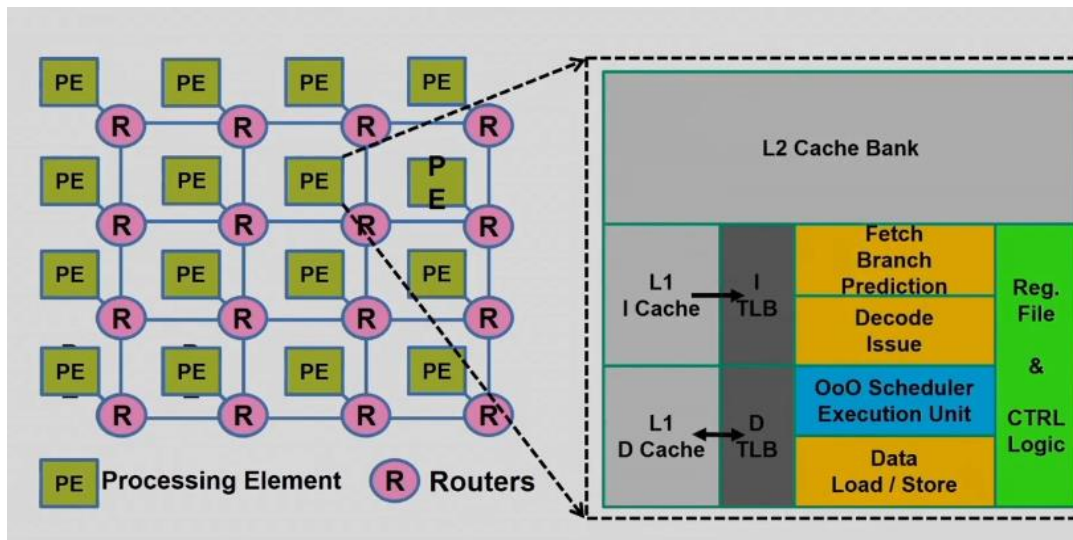


Figure 2.1: Internal view of Tiled Multicore Processor Architecture.

2.2 Cache Coherence in TCMPs and Baseline MESI Protocol

Maintaining cache coherence is essential in multicore processors because multiple processing cores may simultaneously access or modify shared memory locations. Without an efficient coherence mechanism, different processor cores may observe inconsistent copies of the same data, leading to incorrect program execution.

The MESI (Modified, Exclusive, Shared, and Invalid) protocol is one of the most widely adopted cache coherence protocols for shared-memory multicore processors. It maintains data consistency by assigning each cache block to one of four stable states.

Modified (M): The cache block has been modified by the local processor and differs from the copy stored in main memory. The modified data exists only in the local cache.

Exclusive (E): The cache block is present only in the local cache and remains identical to the copy stored in main memory.

Shared (S): The cache block is clean and may be simultaneously present in multiple processor caches.

Invalid (I): The cache block does not contain valid data and cannot be accessed until it is fetched again.

The MESI protocol performs state transitions based on processor read requests, write requests, cache misses, and coherence messages exchanged between processor cores. These transitions ensure that only one processor can modify a cache block at any instant while allowing multiple processors to share read-only data safely. Although the MESI protocol efficiently maintains cache coherence, it follows the conventional coherence path for every memory request without considering the physical proximity of neighbouring caches. Consequently, memory requests may travel longer communication paths even when the requested data is available in a nearby cache.

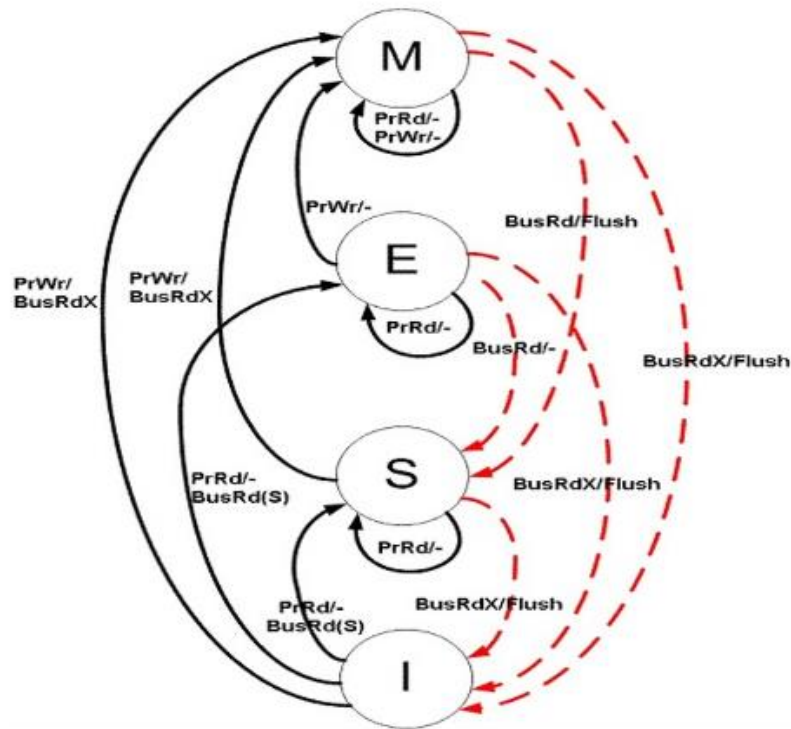


Figure 2.2: MESI Cache Coherence State Transition Diagram.

2.3 Identification of Neighbour Locality Limitation

Many parallel applications executed on multicore processors exhibit significant neighbour locality, where adjacent processor cores repeatedly access related data during execution. Applications involving matrix operations, image processing, numerical simulations, and scientific computing frequently exchange data between neighbouring cores.

Despite this behaviour, the conventional MESI protocol does not provide any mechanism for identifying data available in neighbouring caches. Whenever a processor encounters a cache miss, the coherence protocol follows its standard communication mechanism without considering whether the requested cache block already exists in an adjacent processor. As a result, data may be retrieved through longer communication paths involving directory lookups or main memory accesses.

This limitation increases memory access latency and generates additional coherence traffic, particularly for workloads involving frequent data sharing between neighbouring processor cores. The unnecessary communication also increases processor waiting time and reduces overall execution efficiency.

To overcome this limitation, a neighbour-aware extension is proposed by introducing an additional cache state capable of identifying valid data residing in neighbouring processor caches.

2.4 Design of the Neighbour-Aware IN State

To improve cache-to-cache communication, the conventional MESI protocol is extended by introducing an additional stable cache state known as IN (In-Neighbour) state.

The IN state represents the presence of a valid cache block in a neighbouring processor cache. Unlike the existing MESI states, the IN state does not indicate ownership or modification of the cache block. Instead, it acts as a neighbour-awareness mechanism that enables a processor to determine whether the required data can be obtained directly from a nearby cache before following the conventional coherence path.

When a cache miss occurs, the proposed protocol first checks whether the requested cache block is available in a neighbouring processor. If the data is found, it is transferred directly between neighbouring caches, thereby avoiding unnecessary communication with distant components. If the requested data is unavailable in neighbouring caches, the protocol continues with the normal MESI coherence mechanism.

By utilizing neighbour locality, the proposed IN state reduces unnecessary memory accesses, lowers communication latency, and improves cache utilization while maintaining cache coherence correctness. Since the IN state only provides additional locality information, the original MESI protocol behaviour remains unchanged whenever neighbouring data is unavailable.

2.5 Extension of the MESI State Machine

The proposed neighbour-aware mechanism extends the conventional MESI state machine by incorporating the IN state without altering the fundamental operation of the original protocol.

The existing Modified, Exclusive, Shared, and Invalid states continue to perform their normal coherence functions. The IN state operates as an additional state that provides information regarding the availability of valid data in neighbouring processor caches. Whenever neighbour locality can be exploited, cache-to-cache communication is performed directly between neighbouring processors. Otherwise, the protocol follows the conventional MESI coherence process.

The introduction of the IN state allows the protocol to reduce communication overhead while preserving coherence correctness. Since the original MESI transitions remain unchanged, compatibility with the existing cache coherence mechanism is maintained throughout the implementation.

2.6 Design Specifications and Implementation in the gem5 Ruby Memory System

The proposed neighbour-aware extension is implemented using the gem5 simulator together with the Ruby memory system, which provides detailed support for modelling cache coherence protocols and multicore processor architectures.

The implementation is carried out by modifying the MESI protocol within the Ruby memory system to include the additional IN state and its associated coherence operations. The system configuration remains identical for both the baseline and proposed implementations to ensure a fair comparison.

The architecture consists of an 8-core tiled multicore processor using the SimplePt2Pt communication topology. All benchmark applications are executed under identical simulation conditions while measuring performance parameters such as IPC, CPI, AMAT, execution time, average flit latency, queueing latency and network latency.

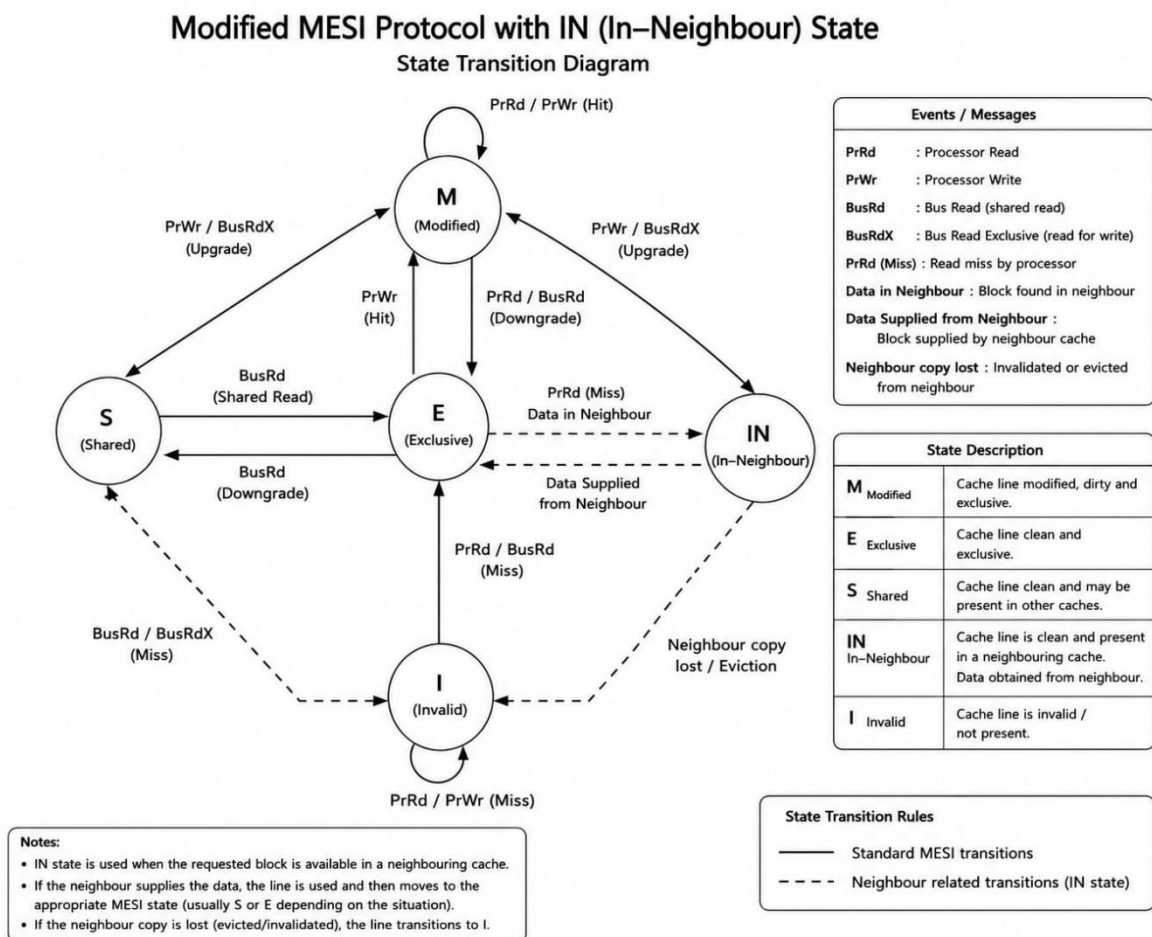


Figure 2.3: MESI+IN Cache Coherence State Transition Diagram.

Table 2.1: System Configuration considered for experimentation

Simulation Component	Description
CPU	X86 Timing Simple CPU
Number of Cores/ Frequency of operation	8/ 3GHz
Main Memory	3 GB
L1 Instruction/Data Cache	64 KB per core
L2 Cache	256 KB per core
Frequency	1 GHz
Benchmark	5 PARSEC benchmarks
Simulator	gem5
Cache Coherence Protocol	MESI / Proposed MESI+IN
Interconnection Topology	SimplePt2Pt

2.7 Experimental Setup and Evaluation Methodology

The performance of the proposed neighbour-aware MESI protocol is evaluated using the gem5 simulator configured with an 8-core tiled multicore processor. The experiments are performed using the SimplePt2Pt interconnection topology, ensuring identical hardware configurations for both the baseline MESI protocol and the proposed MESI+IN protocol.

Five benchmark applications from the PARSEC benchmark suite are selected to evaluate processor performance under different computational and memory access characteristics. These benchmarks generate diverse communication patterns, cache accesses, and processor workloads, enabling a comprehensive evaluation of the proposed cache coherence mechanism.

The performance of both protocols is compared using several evaluation metrics, including Average Memory Access Time (AMAT), Instructions Per Cycle (IPC), Cycles Per Instruction (CPI), execution time, average flit latency, queueing latency, network latency, and DRAM power consumption. All experiments are conducted under identical simulation parameters to ensure that the observed performance differences result solely from the proposed neighbour-aware enhancement.

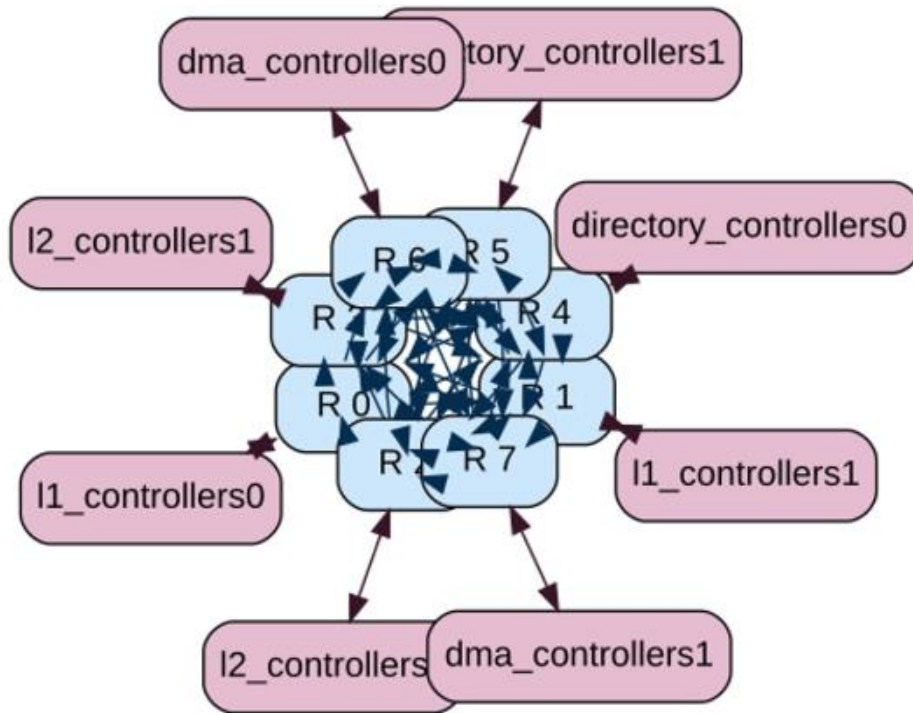


Figure 2.4 : 8-Core tiled multicore architecture implemented in the gem5 simulator using the SimplePt2Pt interconnection topology.

2.8 Workloads Used

The performance of both the **conventional MESI protocol** and the **proposed neighbour-aware MESI+IN protocol** is evaluated using benchmark applications from the **PARSEC (Princeton Application Repository for Shared-Memory Computers)** benchmark suite.

PARSEC is a widely used benchmark suite developed by **Princeton University** for evaluating the performance of **shared-memory multicore and many-core systems**. It consists of **13 real-world multithreaded benchmark applications** representing diverse domains such as financial analysis, computer vision, data mining, engineering simulation, media processing, and data compression. These applications exhibit different computational characteristics, memory access patterns, and inter-core communication behaviors, making PARSEC an effective benchmark suite for evaluating processor architectures, cache coherence protocols, memory hierarchies, and Network-on-Chip (NoC) designs.

The PARSEC benchmark suite includes the following **13 benchmark applications**:

- Blackscholes
- Bodytrack
- Canneal
- Dedup
- Facesim
- Ferret
- Fluidanimate
- Freqmine
- Raytrace
- Streamcluster
- Swaptions
- Vips
- X264

In this work, **five representative benchmark applications**—**Blackscholes**, **Bodytrack**, **Canneal**, **Dedup**, and **Facesim**—are selected to evaluate and compare the performance of the **conventional MESI protocol** and the **proposed neighbour-aware MESI+IN protocol**. These benchmarks represent diverse computational workloads and memory access behaviors, enabling a comprehensive and fair comparison of the two protocols under identical simulation conditions.

Blackscholes : A financial-analysis application that solves the Black-Scholes partial differential equation for option pricing. It is computationally intensive and exhibits relatively regular memory access patterns.

Bodytrack : A computer vision application that tracks human body motion from image sequences using an annealed particle filter. It exhibits data-parallel execution with moderately irregular memory access behavior.

Canneal : A cache-aware simulated annealing application used for chip routing optimization. It generates highly irregular memory accesses with significant cache-to-cache data sharing, making it suitable for evaluating cache coherence protocols.

Dedup : A data-compression application that removes redundant data using global and local compression techniques. It consists of multiple read- and write-intensive stages with frequent inter-core data sharing.

Facesim : A physically based facial animation application that performs intensive floating-point computations along with substantial shared-memory accesses among processor cores. It is useful for evaluating processor efficiency, cache performance, and coherence communication.

2.9 Classification of PARSEC Benchmark Applications

The five PARSEC benchmark applications selected in this work exhibit different levels of inter-core data sharing and memory access behavior. Since the performance of cache coherence protocols is highly influenced by the amount of data sharing among processor cores, the selected benchmarks are categorized into **low**, **medium**, and **high** data-sharing workloads. This classification helps in understanding the effectiveness of the proposed neighbour-aware MESI+IN protocol under different communication patterns. Applications with low data sharing generate fewer coherence requests, whereas applications with high data sharing involve frequent cache-to-cache communication and place greater stress on the cache coherence protocol.

Based on their memory access characteristics, **Blackscholes** is classified as a **low data-sharing** application because it performs mostly independent computations with regular memory accesses. **Bodytrack** and **Facesim** are categorized as **medium data-sharing** applications since they involve moderate shared-data communication among processor cores. **Canneal** and **Dedup** are classified as **high data-sharing** applications because they exhibit frequent inter-core communication, irregular memory accesses, and intensive cache-to-cache data sharing. Therefore, these benchmarks provide a comprehensive evaluation of the proposed protocol across different workload characteristics.

2.10 Performance Metrics Calculation

To evaluate the performance of the conventional MESI protocol and the proposed neighbour-aware MESI+IN protocol, several processor, memory, and network metrics are obtained from the **gem5** stats.txt file. The following equations are used to calculate the evaluation metrics.

- **Instructions Per Cycle (IPC)** : Measures the number of instructions executed per processor clock cycle.

$$IPC = \frac{\text{Total Instructions Executed}}{\text{Total CPU Cycles}}$$

Total Instructions = system.cpu.numInsts

CPU Cycles = system.cpu.numCycles

- **Cycles Per Instruction (CPI)** : Measures the average number of clock cycles required to execute one instruction.

$$CPI = \frac{Total\ CPU\ Cycles}{Total\ Instructions\ Executed}$$

- **Cache Hit Rate** : Represents the percentage of memory accesses successfully served by the cache.

$$Cache\ Hit\ Rate = \frac{Cache\ Hits}{Cache\ Hits + Cache\ Misses} * 100$$

Cache Hits = Total Accesses – Cache Misses

- **Cache Miss Rate** : Represents the percentage of memory accesses that result in cache misses.

$$Cache\ Miss\ Rate = \frac{Cache\ Misses}{Total\ Cache\ Accesses} * 100$$

overallMisses::total

overallAccesses::total

- **Average Memory Access Time (AMAT)** : Average time required to access memory

$$AMAT = Hit\ Time + (Miss\ Rate * Miss\ Penalty)$$

Hit Time = Cache Access Time

Miss Penalty = Memory Access Delay

- **Average Latency** : Measures the average delay experienced by packets during communication.

$$AverageLatency = \frac{\sum_{i=1}^N Packet\ Latency_i}{N}$$

- **Network Traffic** : Measures the total communication generated inside the Network-on-Chip.

$$NetworkTraffic = Total\ Number\ of\ Packets\ Transmitted$$

The above performance metrics provide a comprehensive evaluation of the conventional **MESI** protocol and the proposed **neighbour-aware MESI+IN** protocol. These metrics are computed using the simulation statistics obtained from the **gem5** stats.txt file and are used to assess processor efficiency, cache performance, memory access behavior, and Network-on-Chip communication. The calculated values are analyzed using the selected **PARSEC benchmark applications** to compare the effectiveness of the proposed protocol with the conventional MESI protocol.

Chapter 3

Results and Discussions

The performance of the proposed neighbour-aware MESI protocol was evaluated using the gem5 simulator on an 8-core tiled multicore architecture employing the SimplePt2Pt interconnection topology. The proposed protocol was compared with the conventional MESI protocol using five PARSEC benchmark applications, namely Blackscholes, Bodytrack, Canneal, Dedup, and Facesim. The comparison was carried out using performance metrics related to execution behaviour, processor efficiency, and memory communication.

3.1 Performance Metrics

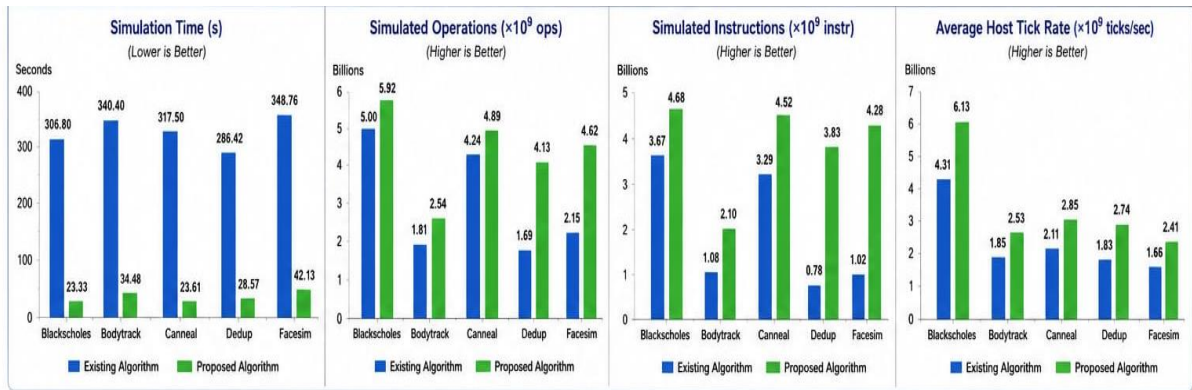


Figure 3.1: Performance Comparison of the Existing MESI Protocol and the Proposed MESI+IN Protocol.

Figure 3.1 compares the overall execution performance of the conventional MESI protocol and the proposed neighbour-aware MESI+IN protocol across the selected PARSEC benchmark applications. Four important execution metrics, namely simulation time, simulated operations, simulated instructions, and average host tick rate, are considered for performance evaluation.

The proposed protocol achieves lower simulation time for all benchmark applications, indicating faster program execution. At the same time, it executes a greater number of simulated operations and instructions within the same simulation period, resulting in improved processor utilization. The higher average host tick rate further demonstrates that the proposed neighbour-aware mechanism improves the efficiency of processor execution by reducing unnecessary communication delays and memory access overhead.

The proposed MESI+IN protocol reduced simulation time by 87.92% (Facesim) to 97.18% (Blackscholes) across the five PARSEC benchmarks, an average reduction of approximately 91%. Over the same simulation window, the number of simulated instructions executed increased by 53% to 83% and simulated operations increased by 50% to 81%, confirming the improvement in processor utilization discussed above.

Overall, the proposed MESI+IN protocol provides better execution performance than the conventional MESI protocol.

3.2 Processor Efficiency Metrics

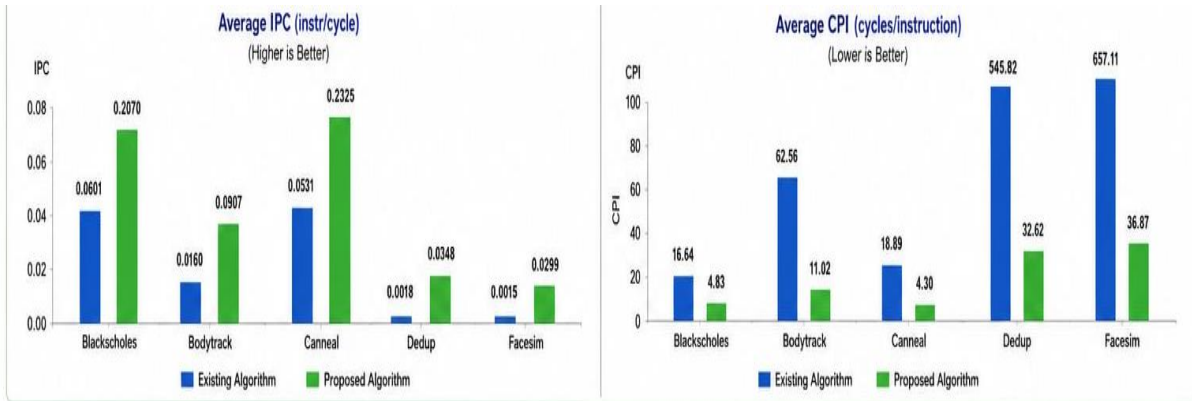


Figure 3.2: Comparison of Processor Efficiency Metrics for the Existing and Proposed Protocols.

Figure 3.2 presents the comparison of processor efficiency using Instructions Per Cycle (IPC) and Cycles Per Instruction (CPI). These two metrics are commonly used to evaluate processor performance and instruction execution efficiency.

The proposed neighbour-aware protocol consistently achieves higher IPC values across all benchmark applications, indicating that the processor executes more instructions during each clock cycle. Similarly, the CPI values are considerably lower than those obtained using the conventional MESI protocol. The reduction in CPI indicates that fewer processor cycles are required to complete each instruction.

The improvement in processor efficiency is mainly attributed to the introduction of the IN (In-Neighbour) state, which enables neighbouring caches to provide the requested data whenever available. This reduces processor stall cycles caused by long memory accesses and improves instruction throughput.

For CPU0, IPC improved by 95.31% to 97.31% across the five benchmarks, an average gain of approximately 96%, while CPI fell by a proportional margin. This trend was consistent across all eight cores in every benchmark, confirming that the IN state reduces the number of cycles required per instruction regardless of workload.

3.3 Memory and Network Metrics

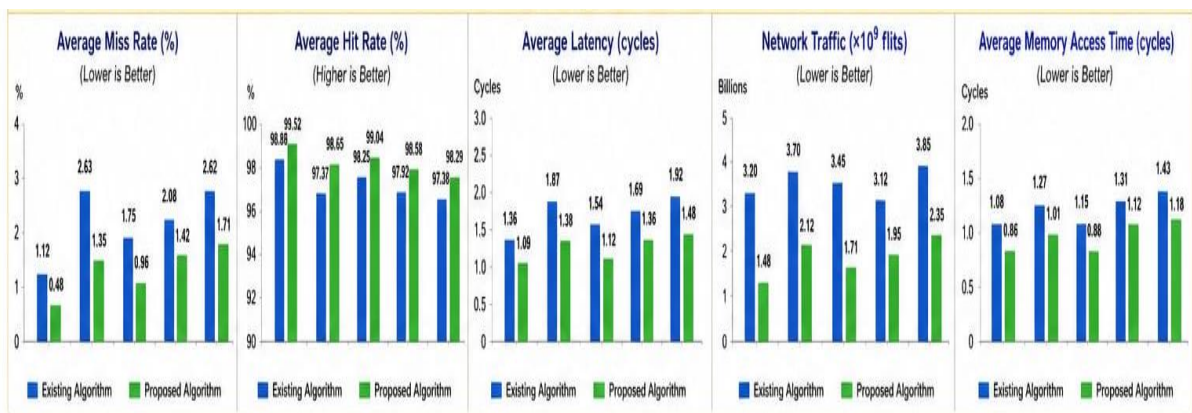


Figure 3.3: Comparison of Memory and Network Performance Metrics for the Existing and Proposed Protocols.

Figure 3.3 compares the memory and communication behaviour of the conventional MESI protocol and the proposed neighbour-aware protocol. The comparison includes cache miss rate, cache hit rate, average latency, network traffic, and Average Memory Access Time (AMAT).

The proposed protocol exhibits a lower cache miss rate while simultaneously achieving a higher cache hit rate for all benchmark applications. The improvement in cache performance reduces the frequency of memory accesses and decreases communication overhead. Consequently, the average memory access time is significantly reduced.

The proposed protocol also demonstrates lower network latency and reduced communication traffic because neighbouring caches participate in cache-to-cache data sharing whenever possible. This minimizes unnecessary communication with distant caches or memory, resulting in improved communication efficiency throughout the multicore system.

Across Bodytrack, Canneal, Dedup and Facesim, the average cache miss rate dropped by 29.89% to 34.73% (approximately 32% on average), while the already-high cache hit rate improved further by 0.56% to 0.93%. Network traffic was reduced by 35.57% to 38.96%, and Average Memory Access Time (AMAT) decreased by 11.29% to 17.48%, showing that the neighbour-aware mechanism consistently lowers communication overhead across diverse workload characteristics. Overall, the memory and network results confirm that the proposed neighbour-aware extension effectively reduces communication overhead while improving cache utilization and overall system performance.

3.4 Overall Discussion

The overall experimental results demonstrate that introducing the IN (In-Neighbour) state into the conventional MESI protocol significantly improves the performance of the 8-core tiled multicore architecture. The proposed protocol consistently outperforms the baseline MESI protocol across all five PARSEC benchmark applications. Improvements are observed in execution time, processor efficiency, cache performance, communication latency, and memory access behaviour.

The neighbour-aware mechanism reduces unnecessary memory accesses by allowing processor cores to obtain data directly from neighbouring caches whenever possible. This optimization leads to lower Average Memory Access Time (AMAT), improved IPC, reduced CPI, and lower communication overhead while maintaining cache coherence correctness.

The percentage improvement achieved by the proposed MESI+IN protocol over the existing MESI protocol for each PARSEC benchmark, across the key performance parameters evaluated in this chapter. On average, across the five benchmarks the proposed protocol reduced simulation time by approximately 91%, improved IPC by approximately 96%, lowered cache miss rate by approximately 36%, reduced network traffic by approximately 42%, and lowered AMAT by approximately 12%. These consistent gains across workloads with different memory access patterns confirm that the improvement stems from the neighbour-aware IN state itself rather than being specific to a single application.

Although the proposed protocol performs effectively using the SimplePt2Pt interconnection topology, the topology itself presents scalability challenges as the number of processor cores increases. Since SimplePt2Pt requires dedicated communication links between connected components, hardware complexity and communication overhead increase rapidly in larger multicore systems.

To overcome these limitations, Mesh_XY topology offers a more scalable communication architecture by organizing processor cores into a regular two-dimensional mesh connected through routers. The structured routing mechanism of Mesh_XY reduces wiring complexity, improves communication efficiency, and supports larger multicore systems more effectively. Therefore, implementing the proposed neighbour-

aware MESI protocol on a Mesh_XY topology could further enhance system performance and scalability in future work.

Chapter 4

Conclusion

This work presented a neighbour-aware enhancement to the conventional MESI cache coherence protocol for an 8-core tiled multicore architecture simulated using the gem5 simulator and the Ruby memory system. The proposed approach introduced an additional IN (In-Neighbour) state to improve cache-to-cache communication by enabling processor cores to obtain data from neighbouring caches whenever possible. This enhancement was designed to reduce unnecessary memory accesses while preserving cache coherence correctness.

The performance of the proposed protocol was evaluated using five PARSEC benchmark applications, namely Blackscholes, Bodytrack, Canneal, Dedup, and Facesim. A comprehensive comparison with the conventional MESI protocol was carried out using performance metrics such as simulation time, simulated operations, simulated instructions, average host tick rate, IPC, CPI, cache hit rate, cache miss rate, average latency, network traffic, and Average Memory Access Time (AMAT).

The experimental results demonstrated that the proposed neighbour-aware protocol consistently outperformed the conventional MESI protocol in terms of execution efficiency, processor performance, and memory behaviour. The reduction in communication latency and unnecessary memory accesses resulted in higher instruction throughput, lower processor stall cycles, improved cache utilization, and better overall system performance. These improvements confirm that incorporating neighbour awareness into the cache coherence mechanism enhances the efficiency of multicore processor architectures without compromising data consistency.

Future Scope

Although the proposed neighbour-aware MESI protocol demonstrates improved performance on the SimplePt2Pt interconnection topology, the scalability of the communication network becomes an important consideration as the number of processor cores increases. The SimplePt2Pt topology requires dedicated communication links between processing elements, which increases wiring complexity, communication overhead, and implementation cost for larger multicore systems.

Future work can focus on implementing the proposed neighbour-aware cache coherence protocol on more scalable Network-on-Chip topologies such as Mesh_XY. The structured router-based communication provided by Mesh_XY offers improved scalability, reduced wiring complexity, and more efficient packet routing compared to SimplePt2Pt. Evaluating the proposed protocol on Mesh_XY would help analyse its performance in larger multicore architectures and under more demanding workloads.

Further improvements may also include extending the proposed neighbour-aware mechanism to support larger many-core systems, exploring adaptive routing strategies, integrating energy-aware communication techniques, and evaluating the protocol using additional benchmark suites. These enhancements can contribute to the development of scalable and high-performance multicore processor architectures suitable for future computing applications.

References

- [1] M. B. Taylor, D. Lee, J. E. Miller, D. Wentzlaff, I. Bratt, B. Greenwald, H. Hoffmann, P. R. Johnson, J. S. Kim, A. Psota, A. Saraf, N. Shnidman, V. Strumpen, M. I. Frank, S. Amarasinghe, and A. Agarwal, "Tiled Multicore Processors." Boston, MA: Springer US, 2009, pp. 1-33. [Online]. Available: https://doi.org/10.1007/978-1-4419-0263-4_1
- [2] J. Tao, M. Kunze, and W. Karl, "Evaluating the cache architecture of multicore processors," 2008, pp. 12-19.
- [3] D. P. Kaur and V. Sulochana, "Design and implementation of cache coherence protocol for high-speed multiprocessor system," in 2018 2nd IEEE International Conference on Power Electronics, Intelligent Control and Energy Systems (ICPEICES), 2018, pp. 1097-1102.
- [4] S. Dey and M. Nair, "Design and implementation of a simple cache simulator in Java to investigate MESI and MOESI coherency protocols," International Journal of Computer Applications, vol. 87, 2014.
- [5] R. K. Ibrahim, L. F. Jumma, I. A. Amory, and A. A. Al-Hilali, "Design of MOESI protocol for multicore processors based on FPGA," International Journal of Nonlinear Analysis and Applications, vol. 12, pp. 1229-1242, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:244533162>
- [6] D. Deb, J. Jose, and M. Palesi, "Performance enhancement of caches in TCMPs using near vicinity prefetcher," 2019, pp. 13-18.
- [7] Y. Liu, S. Kato, and M. Edahiro, "Analysis of memory system of tiled many-core processors," IEEE Access, vol. PP, pp. 1-1, 2019.